

```

"""
StegSim governance.py - Commit-time admissibility evaluation engine.
This is the core of StegSim: every proposed transition is evaluated
BEFORE it commits. Local policy check alone is insufficient.
Implements the viability-aware governance logic from the VBDS spec.
Compatible with GCAT/BCAT formalism (calls triad_validator if available).
"""

import math
import hashlib
import json
from typing import Optional
from .model import (
    AgentState, SystemState, Proposal, EvaluationResult,
    ViabilityMargin, ACTION_TO_TRANSITION_CLASS, TRANSITION_CLASSES,
    state_hash,
)

def _sha(s: str) -> str:
    return "sha256:" + hashlib.sha256(s.encode()).hexdigest()

class GovernanceEngine:
    """
    Evaluates every proposal through four gates:
    1. local_policy_valid      - does the agent have standing locally?
    2. authority_valid        - is authority current, not stale?
    3. viability_preserved    - does the transition keep V above threshold?
    4. legitimacy_capacity_ok  - does autonomy stay within Lambda(x)?

    All four must pass for ALLOW.
    Local-only systems check gate 1 only - that's the baseline comparison.
    """

    def __init__(self, cfg):
        self.cfg = cfg
        self.thresholds = cfg.thresholds
        self.weights = cfg.weights_dict()
        self._triad = self._load_triad()

    def _load_triad(self):
        """Try to import existing triad_validator from GCAT-BCAT-Engine/workflows
        try:

```

```

import sys, os
workflows_path = os.environ.get("STEGVERSE_WORKFLOWS_PATH", "")
if workflows_path:
    sys.path.insert(0, workflows_path)
from triad_validator import TriadValidator
return TriadValidator()
except ImportError:
    return None

# — Main evaluation —————

def evaluate(self, proposal: Proposal, state: SystemState) -> EvaluationResult:
    agent = state.agents.get(proposal.agent_id)
    if not agent:
        return self._fail_closed(proposal, state, "Agent not found in system")

    env = state.environment

    # Gate 1: Local policy
    local_ok, local_reason = self._check_local_policy(agent, proposal, env)

    # Gate 2: Authority validity (current state, not stale)
    auth_ok, auth_reason, auth_staleness = self._check_authority(agent, env)

    # Gate 3: Viability preservation (project post-transition viability)
    viability_before = state.viability.composite
    projected_v, viability_ok, viab_reason = self._check_viability_preservation(
        agent, proposal, state, env
    )

    # Gate 4: Legitimacy capacity (GCAT Lambda check)
    legit_ok, legit_reason, lambda_val, autonomy_projected = self._check_legitimacy(
        agent, proposal
    )

    # Aggregate decision
    local_global_disagree = local_ok and not (auth_ok and viability_ok and le

    if not local_ok:
        decision = "DENY"
        reason = f"Local policy failed: {local_reason}"
    elif not auth_ok:
        decision = "DENY"
        reason = f"Authority invalid under current state: {auth_reason}"
    elif not legit_ok:
        decision = "DENY"
        reason = f"Legitimacy capacity exceeded: {legit_reason}"

```

```

elif not viability_ok:
    # Check if it's a hard fail or just a warning flag
    if projected_v < self.thresholds.inadmissible:
        decision = "FAIL_CLOSED"
        reason    = f"Projected viability {projected_v:.3f} crosses inadmi
    else:
        decision = "FLAG"
        reason    = f"Projected viability {projected_v:.3f} in warning zon
else:
    decision = "ALLOW"
    reason    = "All governance gates passed."

# Projected state hash
projected_agent_dict = dict(agent.to_dict())
projected_agent_dict["autonomy"] = autonomy_projected
projected_hash = state_hash({"agent": projected_agent_dict, "step": state

return EvaluationResult(
    proposal_id=proposal.proposal_id,
    step=proposal.step,
    agent_id=proposal.agent_id,
    decision=decision,
    reason=reason,
    local_policy_valid=local_ok,
    authority_valid_current_state=auth_ok,
    viability_preserved=viability_ok,
    legitimacy_capacity_ok=legit_ok,
    viability_before=viability_before,
    viability_after_projected=projected_v,
    viability_delta=projected_v - viability_before,
    authority_staleness=auth_staleness,
    legitimacy_capacity=lambda_val,
    autonomy_projected=autonomy_projected,
    local_global_disagree=local_global_disagree,
    state_hash_before=proposal.state_hash_before,
    state_hash_after_projected=projected_hash,
)

```

— Gate 1: Local policy —————

```

def _check_local_policy(self, agent: AgentState, proposal: Proposal, env) ->
    """
    Local policy: agent has minimum trust and authority to act.
    This is what a local-only system checks – necessary but not sufficient.
    """
    if agent.trust < 0.1:
        return False, f"Trust {agent.trust:.3f} below minimum 0.10"

```

```

    if agent.authority < 0.05:
        return False, f"Authority {agent.authority:.3f} below minimum 0.05"
    if agent.resources < 0.01 and proposal.action_type == "transfer_resources":
        return False, "Insufficient resources for transfer"
    tc = ACTION_TO_TRANSITION_CLASS.get(proposal.action_type, "T-000")
    tclass = TRANSITION_CLASSES.get(tc, {})
    if tclass.get("consequence") == "critical" and agent.authority < 0.6:
        return False, f"Critical transition {tc} requires authority >= 0.60"
    return True, "ok"

# — Gate 2: Authority validity —————

def _check_authority(self, agent: AgentState, env) -> tuple:
    """
    Authority must be re-derived from current state, not inherited from stale
    This is the key gate that local-only systems miss.
    """
    max_lag = self.thresholds.max_acceptable_policy_lag
    lag = max(0, env.policy_version - agent.policy_snapshot_version)
    staleness = lag / max_lag if max_lag > 0 else 0.0

    if lag > max_lag:
        return False, f"Policy lag {lag} exceeds max {max_lag} – authority is

    if agent.trust < 0.15 and agent.authority > 0.5:
        return False, f"Trust {agent.trust:.3f} too low to support authority

    # Authority must scale with current trust (not historical trust)
    max_authority_for_trust = agent.trust * 1.2 + 0.2
    if agent.authority > max_authority_for_trust:
        return False, f"Authority {agent.authority:.3f} exceeds trust-scaled

    return True, "ok", staleness

# — Gate 3: Viability preservation —————

def _check_viability_preservation(
    self, agent: AgentState, proposal: Proposal, state: SystemState, env
) -> tuple:
    """
    Project post-transition viability.
    DENY/FLAG if transition crosses warning threshold.
    """
    # Estimate post-transition component changes
    action = proposal.action_type
    mag = proposal.action_magnitude
    gv = env.policy_version

```

```

delta_trust      = 0.0
delta_authority  = 0.0
delta_resources  = 0.0
delta_freshness  = 0.0
delta_mutation   = 0.0
delta_frag       = 0.0

if action == "expand_authority":
    delta_authority = mag * 0.3
    delta_mutation  = mag * 0.1
elif action == "transfer_resources":
    delta_resources = -mag * 0.3
elif action == "update_policy":
    delta_freshness = (1.0 - agent.policy_freshness(gv)) * 0.8
elif action == "assert_trust":
    delta_trust = mag * 0.1

# Projected components
n_agents = len(state.agents)
mean_trust      = state.mean_trust() + delta_trust / n_agents
mean_authority  = state.mean_authority() + delta_authority / n_agents
mean_freshness  = state.mean_policy_freshness() + delta_freshness / n_agents
mutation_p      = env.mutation_pressure + delta_mutation / n_agents
frag           = env.fragmentation + delta_frag / n_agents
resource_bal    = 1.0 - env.resource_scarcity

components = {
    "trust_continuity": float(max(0.0, min(1.0, mean_trust))),
    "authority_coherence": float(max(0.0, min(1.0, mean_authority))),
    "resource_balance": float(max(0.0, min(1.0, resource_bal))),
    "policy_freshness": float(max(0.0, min(1.0, mean_freshness))),
    "mutation_pressure": float(max(0.0, min(1.0, mutation_p))),
    "fragmentation": float(max(0.0, min(1.0, frag))),
}

projected = ViabilityMargin.compute(self.weights, components)
pv = projected.composite

current_v = state.viability.composite
ok = pv >= self.thresholds.degraded

reason = ""
if not ok:
    if pv < self.thresholds.inadmissible:
        reason = f"Transition would push viability to {pv:.3f} (inadmissible)"
    else:

```

```

        reason = f"Transition would push viability to {pv:.3f} (warning z

return pv, ok, reason

# — Gate 4: Legitimacy capacity —————

def _check_legitimacy_capacity(self, agent: AgentState, proposal: Proposal) -
    """
    GCAT legitimacy invariant: autonomy must not exceed Lambda(x).
    Lambda(x) = K * g^alpha * c^beta * t^gamma
    """
    t = self.thresholds
    g = agent.alignment
    c = agent.authority
    tr = agent.trust
    a = agent.autonomy

    lambda_val = (
        t.legitimacy_k
        * (g ** t.legitimacy_alpha)
        * (c ** t.legitimacy_beta)
        * (tr ** t.legitimacy_gamma)
    )

    # Project post-transition autonomy
    action = proposal.action_type
    mag = proposal.action_magnitude
    a_projected = a
    if action == "expand_authority":
        a_projected = min(1.0, a + mag * 0.4)

    threshold = lambda_val * t.autonomy_excess_factor
    ok = a_projected <= threshold

    reason = ""
    if not ok:
        reason = f"Projected autonomy {a_projected:.3f} exceeds capacity Lamb

    return ok, reason, lambda_val, a_projected

# — Fail-closed fallback —————

def _fail_closed(self, proposal: Proposal, state: SystemState, reason: str) -
    v = state.viability.composite
    ph = state_hash({"fail": True, "step": state.step})
    return EvaluationResult(
        proposal_id=proposal.proposal_id,

```

```

        step=proposal.step,
        agent_id=proposal.agent_id,
        decision="FAIL_CLOSED",
        reason=reason,
        local_policy_valid=False,
        authority_valid_current_state=False,
        viability_preserved=False,
        legitimacy_capacity_ok=False,
        viability_before=v,
        viability_after_projected=v,
        viability_delta=0.0,
        authority_staleness=1.0,
        legitimacy_capacity=0.0,
        autonomy_projected=0.0,
        local_global_disagree=False,
        state_hash_before=proposal.state_hash_before,
        state_hash_after_projected=ph,
    )

```

— Local-only baseline (for comparison) —————

```

def evaluate_local_only(self, proposal: Proposal, state: SystemState) -> str:
    """
    Evaluate using ONLY local policy (gate 1).
    Used to compute local_global_disagreement_rate.
    Returns 'ALLOW' or 'DENY'.
    """
    agent = state.agents.get(proposal.agent_id)
    if not agent:
        return "DENY"
    local_ok, _ = self._check_local_policy(agent, proposal, state.environment)
    return "ALLOW" if local_ok else "DENY"

```